

IT3105 AI Programming Project 1 - JAX Controller

Liam Robert Shepard

29.01.26

This report describes the runs carried out by the JAX control system implemented for IT3105 and the Cruise Control problem which was not included in the project description.

Cruise Control Problem

For the custom control problem I chose to model the cruise control of a car. Specifically, I chose to control the force outputted by the engine to achieve a given speed in the presence of drag and noise introduced by random hills. The mathematical description follows below.

For this model we assume three forces and only keep track of the components parallel to the ground:

- Engine output: U (acting in the positive direction)
- Drag: kv (acting in negative direction)
- Gravity: $mg \sin(\alpha)$ (direction dependant on α)

where U is the controller signal, k is the drag coefficient, v is the cars velocity, m is the cars mass, g is the gravitational acceleration, and α is the angle of the slope. Positive α represents an upwards slope.

Newtons first law: $\sum F = ma$

We thus get: $ma = mv' = F = U - kv - mg \sin(\alpha)$

Assuming $m = 1$, this simplifies to: $\mathbf{v'} = U - kv - g \sin(\alpha)}$

To introduce noise we add a randomly sampled $D = \alpha'$, so at each timestep $\alpha \pm D$. To simplify the model we impose minimum and maximum limits on α , and a zero lower bound on the car's velocity.

Thus, at each timestep we compute:

1. $\alpha \ += D$ (with constraints)
2. $v' = U - kv - g \sin(\alpha)$
3. $v \ += v' * dt$ (with zero lower bound)

The controller's goal becomes to keep v at a target speed by adjusting U .

Runs

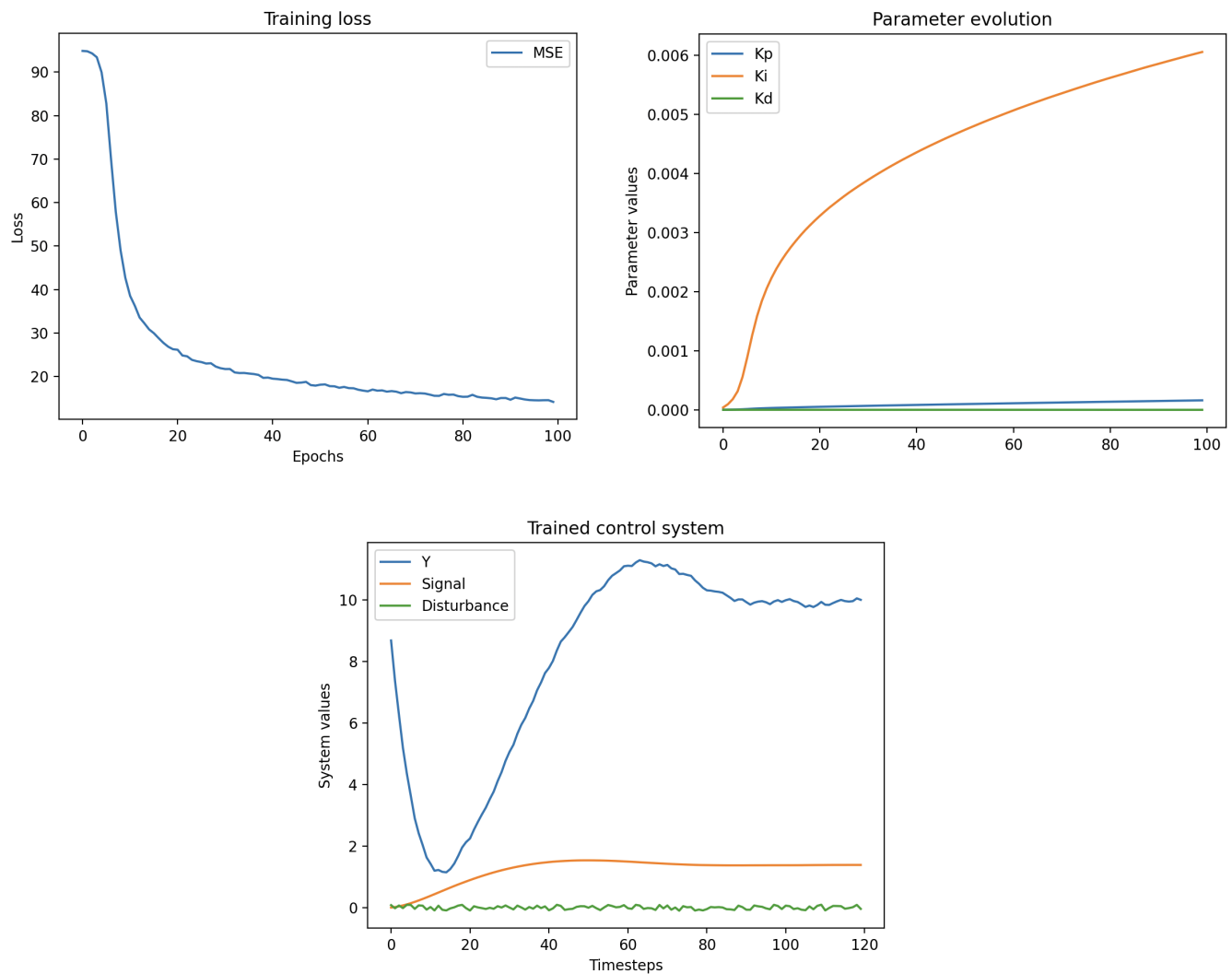
Run 1

Controller: Classic PID

Plant: Bathtub

Parameters:

epochs	timesteps	learning rate	Y0	min_D	max_D	A	C
100	120	1e-8	10	-0.1	0.1	1	0.1



MSE: 4.2637424

From the loss plot we can see that the PID coefficients improved slowly in the start, followed by a phase of rapid improvement, before another longer phase of slow improvements. The parameter

evolution plot shows that the rapid improvement was caused by a sharp change in the integral coefficient. Extrapolating the parameter trend suggests that improvements could be made by increasing the integral coefficient slightly, and the proportional coefficient even more slightly. However looking at the loss plot again, it seems like the potential improvements from further epochs would likely be negligible. From the plot of the trained control system, we can see that after an initial dip in the water level (Y), the control signal increases to get the water level back to its initial level. We can also see that the system is robust to the disturbances.

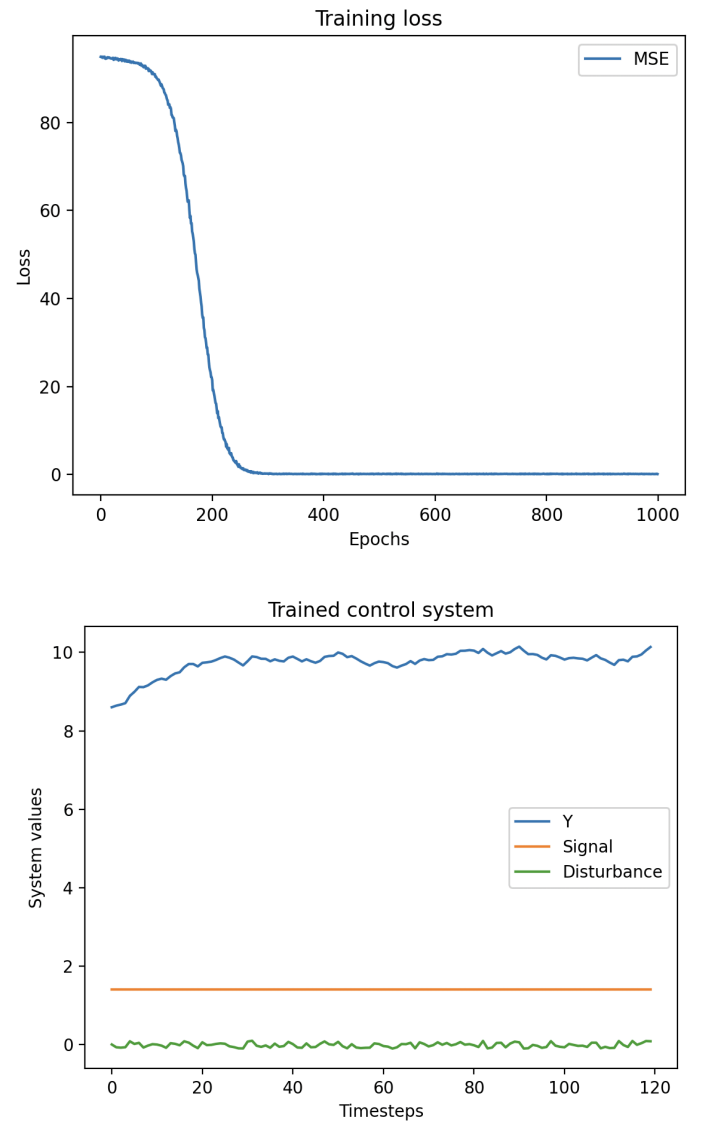
Run 2

Controller: NN PID

Plant: Bathtub

Parameters:

epochs	timesteps	learning rate	Y0	min_D	max_D	A	C	NN Hidden Layers	NN init range
1000	120	1e-4	10	-0.1	0.1	1	0.1	5, ReLU, 2, ReLU	0



MSE: 0.12235692

Similarly to the classic PID controller, the NN pid controller starts off with slow improvements, before a phase of rapid improvement, before another longer convergent phase. Interpreting the parameters is harder in this case than with the classic PID, but the NN may have found a similarly appropriate use of the error integral. Notably, the NN achieved drastically better MSE than the classic PID, likely by utilizing some non linear relations. However it needed many more epochs to converge. The improved performance can be seen in the plot of the simulated trained control system.

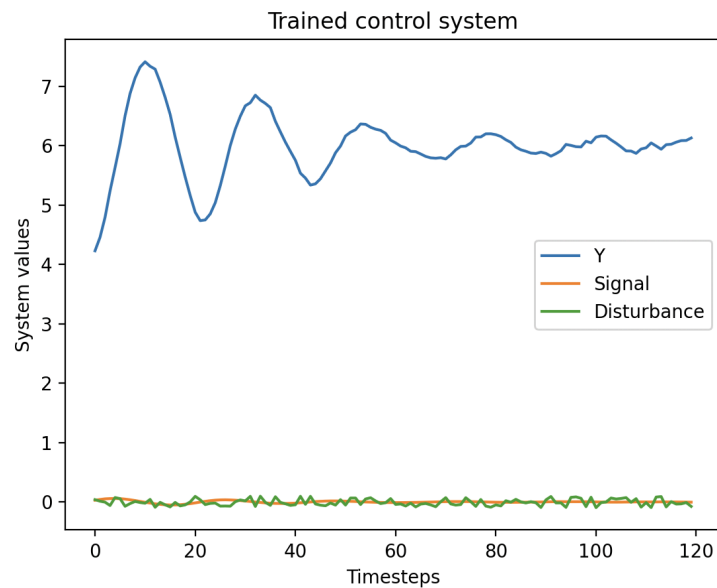
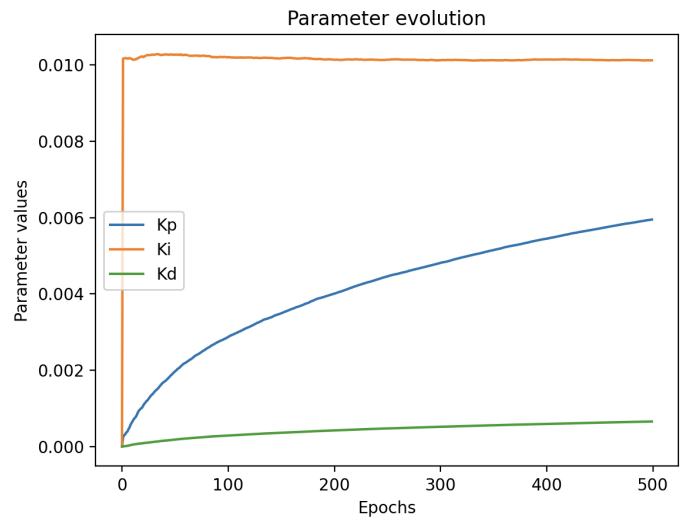
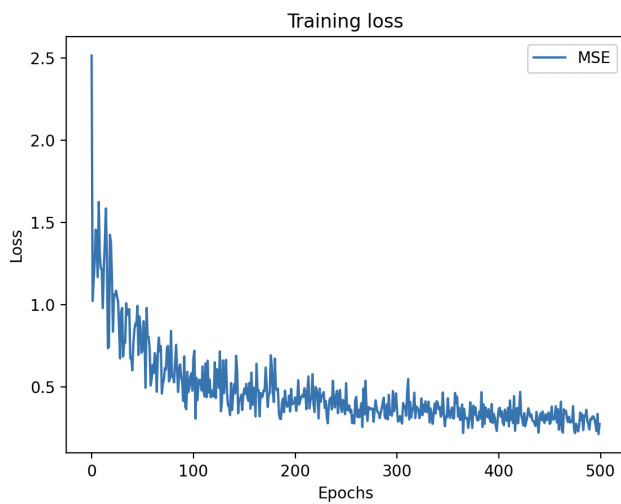
Run 3

Controller: Classic PID

Plant: Cournot Competition

Parameters:

epochs	timesteps	learning rate	Y0	min_D	max_D	max_P	marginal_C
500	120	1e-7	6	-0.1	0.1	10	0.5



MSE: 0.27547768

The controller immediately improved by adjusting its integral coefficient. Subsequent improvements were driven by incremental changes to the proportional and derivative coefficients. The jagged loss curve is likely due to the relatively large variations in disturbances. We can see that over the timesteps, the controller was able to stabilize profit to the desired level, in spite of disturbances.

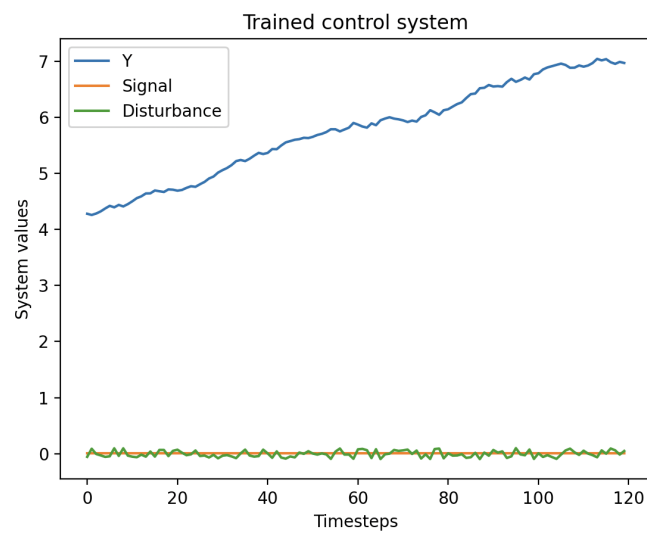
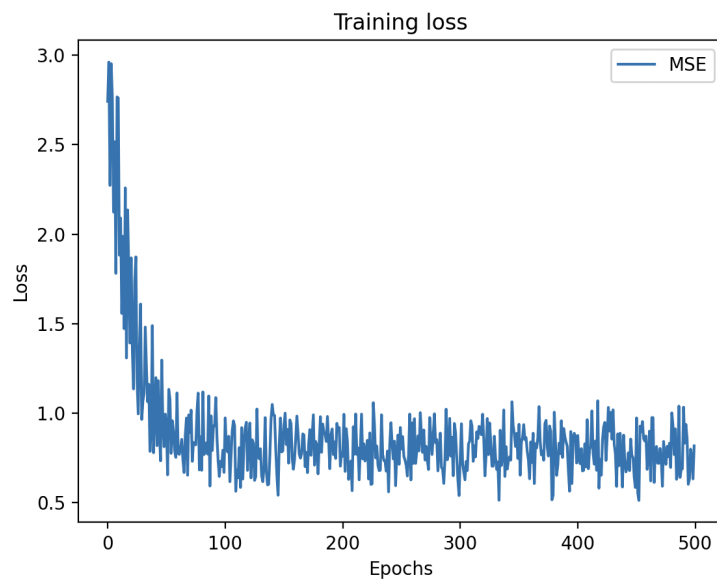
Run 4

Controller: NN PID

Plant: Cournot Competition

Parameters:

epochs	timesteps	learning rate	Y0	min_D	max_D	max P	marginal C	NN Hidden Layers	NN init range
500	120	3e-8	6	-0.1	0.1	10	0.5	5, tanh, 2, sigmoid	0



MSE: 0.8181418

From the loss plot we can see that in this case the NN converged quicker than the classic PID. However, the NN achieved a lower MSE than the classic PID. Looking at the simulated plot, we can see that the performance difference is due to it simply fitting an approximately straight line to average around the desired profit level, instead of stabilizing at it. Trying a few different NN configurations all resulted in this straight line. A possible explanation for this is that there is a local minima that utilizes some nonlinearities, which does not exist in the classic PIDs case. However, I am not sure how plausible this explanation is since the signal seems to be constant and should therefore be reproducible by the classic PID.

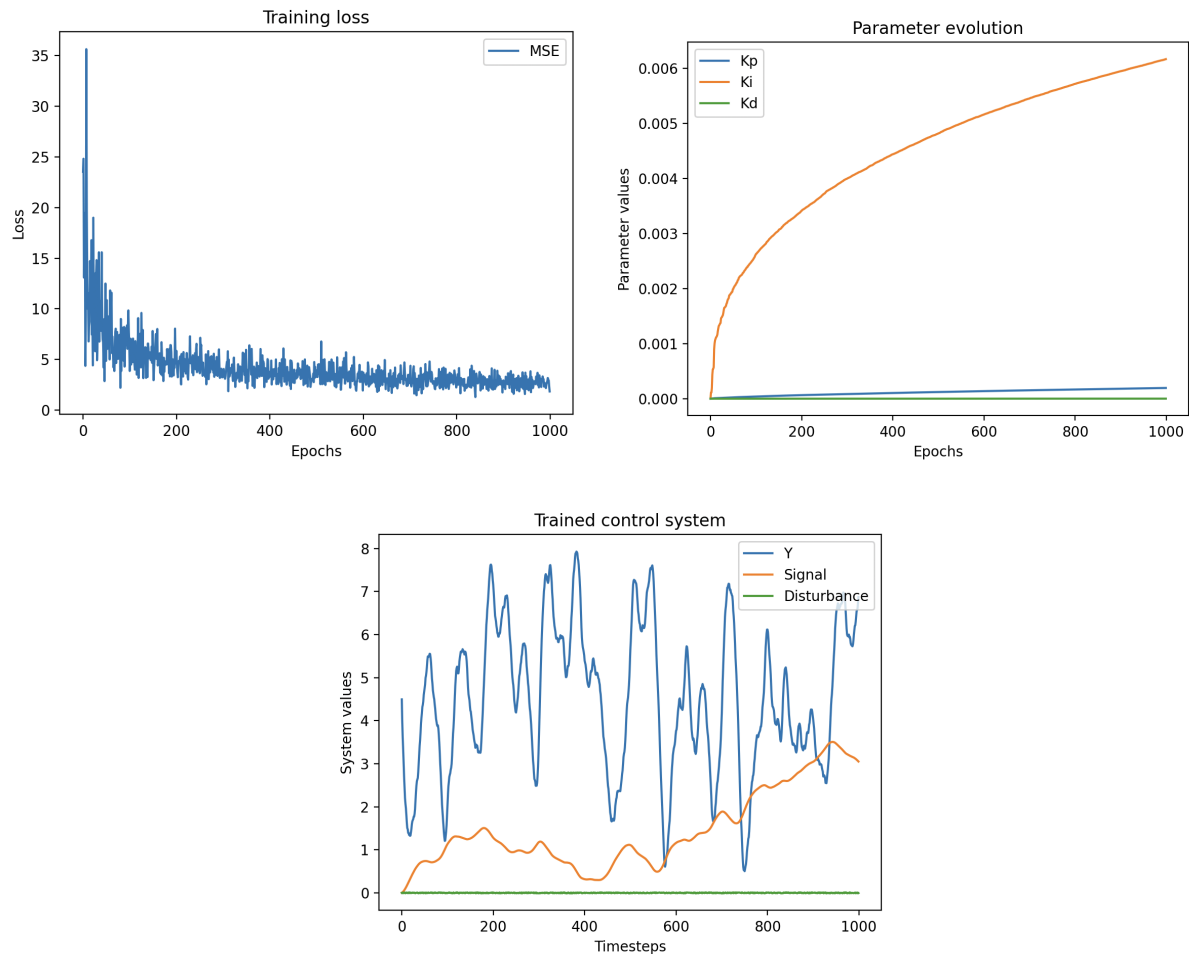
Run 5

Controller: Classic PID

Plant: Cruise Control

Parameters:

epochs	timesteps	learning rate	Y0	min_D	max_D	drag_k	min_a	max_a
1000	1000	5e-9	5	-0.01	0.01	0.1	$-\pi/4$	$\pi/4$



MSE: 1.8512295

Like the two previous problems, the classic PID made quick improvements by adjusting its integral coefficient. Following this was a phase of slower improvement where the integral coefficient and proportional coefficient were gradually increased. The spikes in the loss plot can again be explained by the variance in the disturbances. Looking at the simulated system we can see that the car experiences large disturbances, but that the controller is able to keep it around its desired speed.

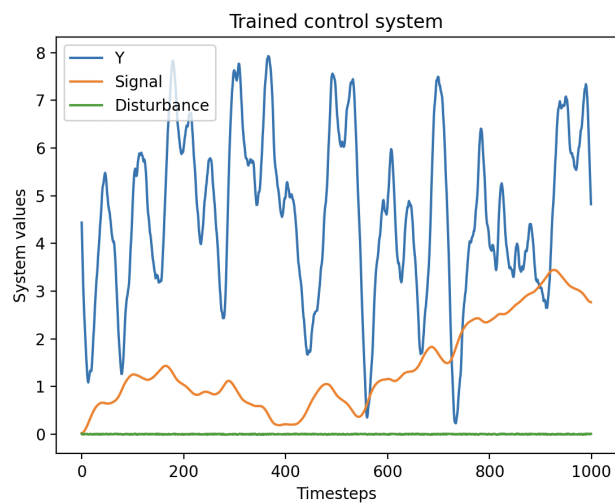
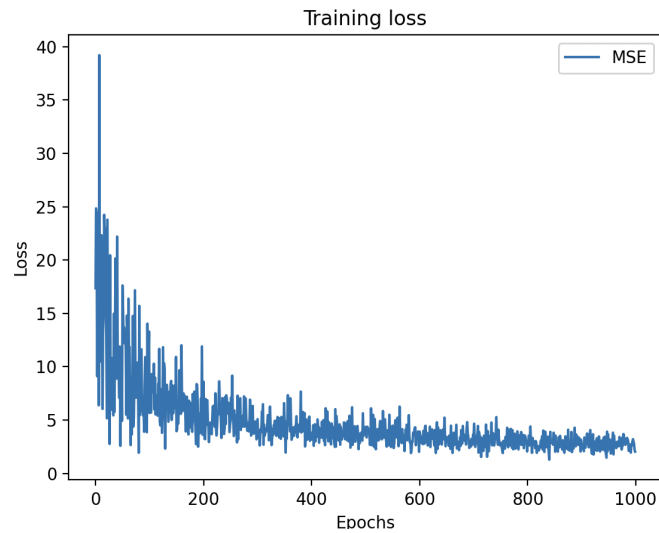
Run 6

Controller: NN PID

Plant: Cruise Control

Parameters:

epochs	timesteps	learning rate	Y0	min_D	max_D	drag_k	min a	max_a	NN hidden layers	NN init range
1000	1000	1e-6	5	-0.01	0.01	0.1	$-\pi/4$	$\pi/4$	3, ReLU	$[-0.1, 0.1]$



MSE: 0.17465686798095703

The NNs loss curve is similar to the classic PIDs, suggesting that it initially found the similar gains from utilizing the integral term. However, we also see that the NN made steadier progress

past this initial phase, achieving a 10x improvement over the classic PID. However, this improvement is not obvious from the simulation plot, where the car appears to experience similar deviations from its target speed as with the classic PID, but this could just be a non-representative example.